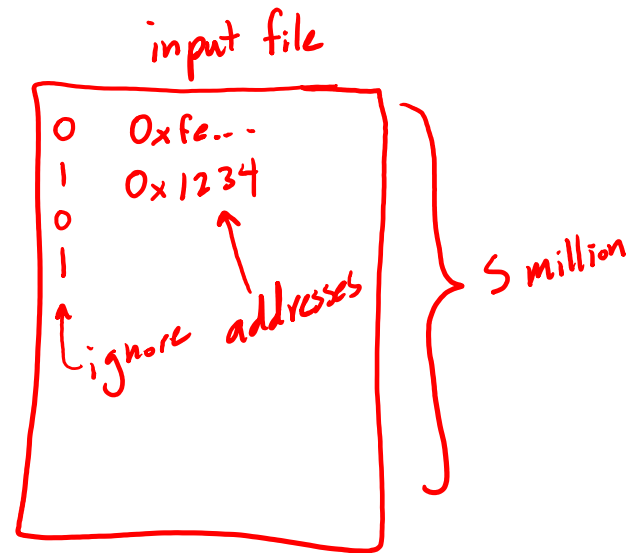


## Lab 6

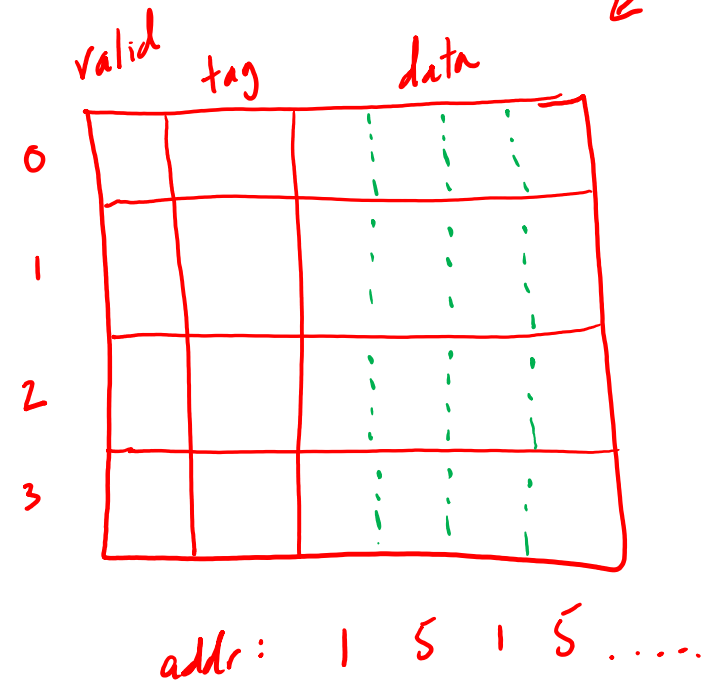
- run all 7 caches at the same time
- code should take less than 10 seconds

- Cache HW on  
Canvas due Friday  
(submit PDF)



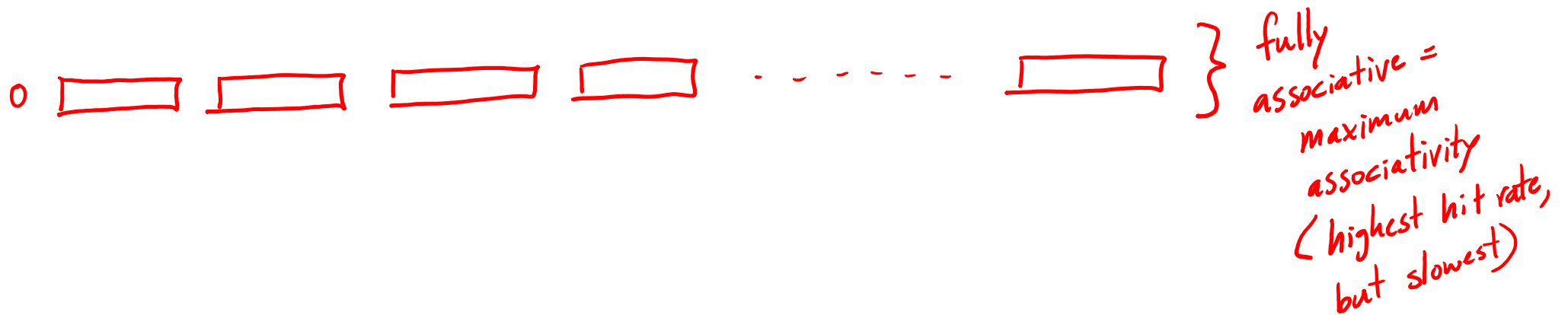
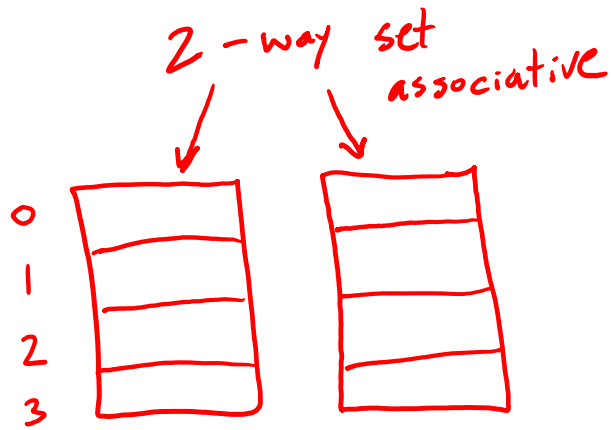
# Where to place a block in the cache

- Direct mapped:



1-way  
(direct  
mapped)

# Associativity → more than 1 location at a given index



# Example

2-way set associative cache

Index = Addr. % 4

|   |                     |          |
|---|---------------------|----------|
| 0 | 4 [data]            |          |
| 1 | <del>9</del> [data] | 5 [data] |
| 2 | 2 [data]            |          |
| 3 |                     |          |

On lookup:

1. check all the ways for a match
- if a miss, then fill in an empty way

$$1 = 0 \text{ } \boxed{01}$$

Addresses:

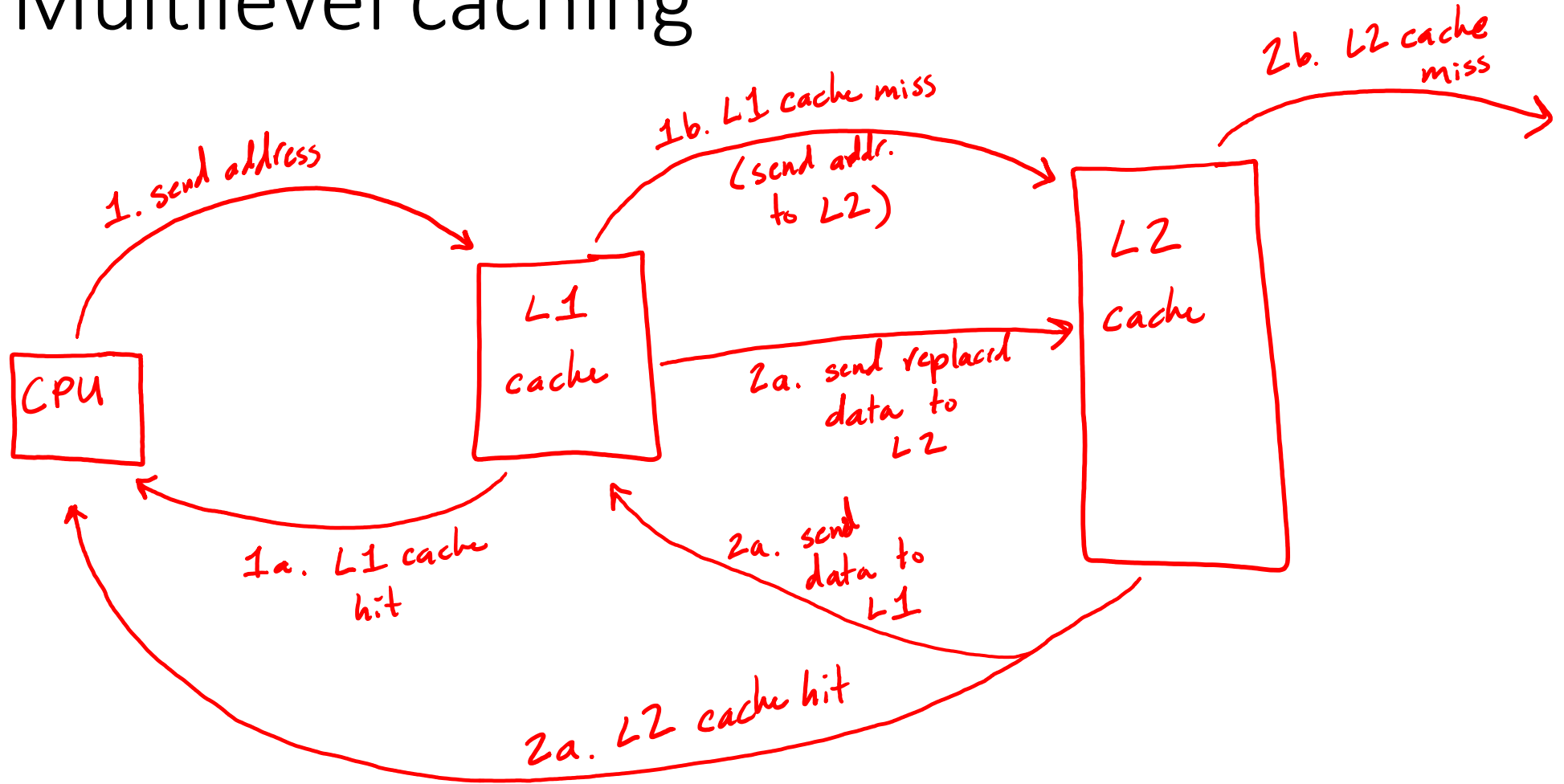
$$5 = 1 \text{ } \boxed{01}$$

|        |   |   |   |   |   |   |   |   |   |   |   |   |   |
|--------|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Addr:  | 1 | 4 | 5 | 2 | 1 | 4 | 5 | 6 | 1 | 4 | 5 | 2 | 9 |
| Index: | 1 | 0 | 1 | 2 | 1 | 0 | 1 | 2 | 1 | 0 | 1 | 2 | 1 |
|        | M | M | M | M | H | H | H | M | H | H | H | H | M |

# Replacement policy

- LRU: *Least Recently Used* - replace the block accessed longest ago
- Random: *randomly select a way to replace*

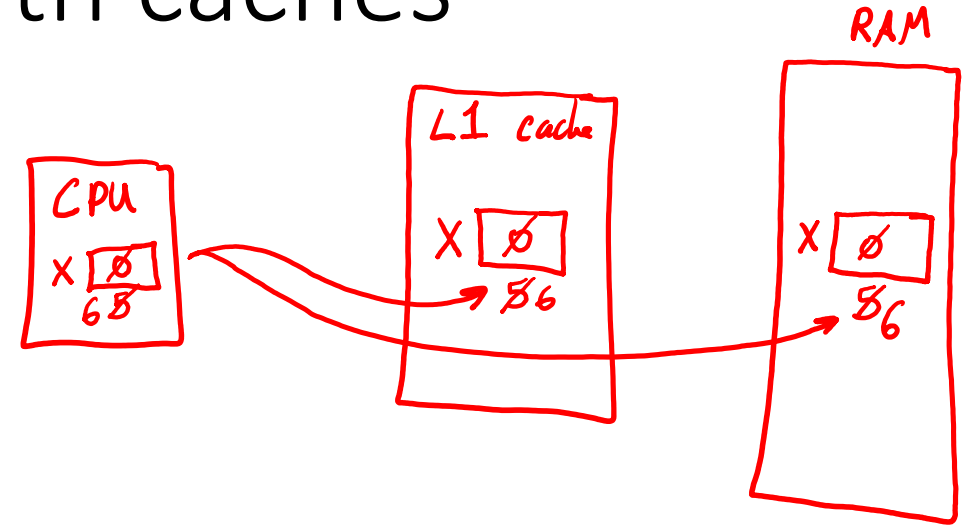
# Multilevel caching



# Writing to memory with caches

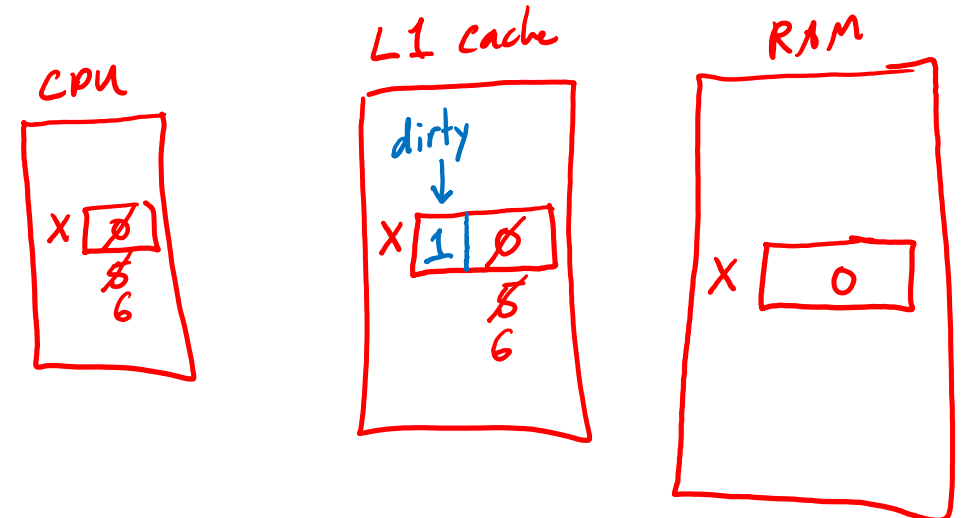
```
int X = 0;  
Z = X + 1;  
X = 5;  
X = 6;
```

- Write-through – any memory write is sent to RAM (always)



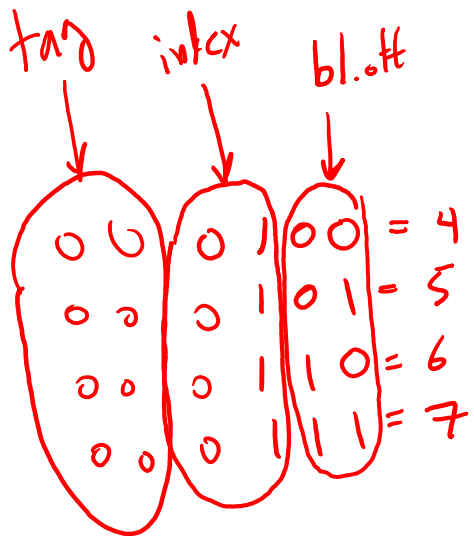
- Write-back – writes the RAM only when a modified cache will be replaced

– the dirty bit indicates a cache line has been modified

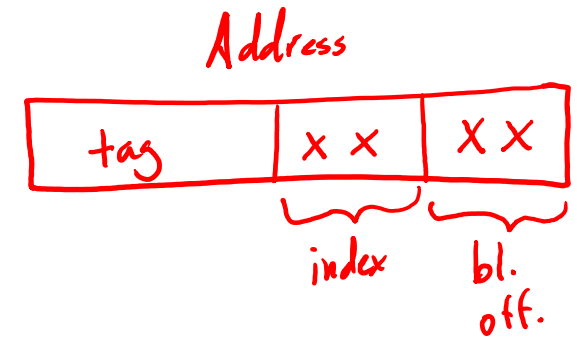


1-way  
Show the hits and misses for a direct-mapped cache with a block length of 4-words and a total size of 16 words. The cache is word addressable (there is no byte-offset).

Addresses:  $\overset{M}{1} \ \overset{M}{4} \ \overset{M}{8} \ \overset{H}{5} \ \overset{M}{20} \ \overset{M}{17}$   
 Bl. off:  $\overset{1}{1} \ \overset{0}{0} \ \overset{0}{0} \ \overset{1}{1} \ \overset{0}{0} \ \overset{1}{1}$   
 Index:  $\overset{0}{0} \ \overset{1}{1} \ \overset{2}{2} \ \overset{1}{1} \ \overset{1}{1} \ \overset{0}{0}$   
 block offset



| index | tag | 0                          | 1                          | 2                          | 3                          |
|-------|-----|----------------------------|----------------------------|----------------------------|----------------------------|
| 0     |     | <del>0</del> <sub>16</sub> | <del>1</del> <sub>17</sub> | <del>2</del> <sub>18</sub> | <del>3</del> <sub>19</sub> |
| 1     |     | <del>4</del> <sub>20</sub> | <del>5</del> <sub>21</sub> | <del>6</del> <sub>22</sub> | <del>7</del> <sub>23</sub> |
| 2     |     | 8                          | 9                          | 10                         | 11                         |
| 3     |     |                            |                            |                            |                            |



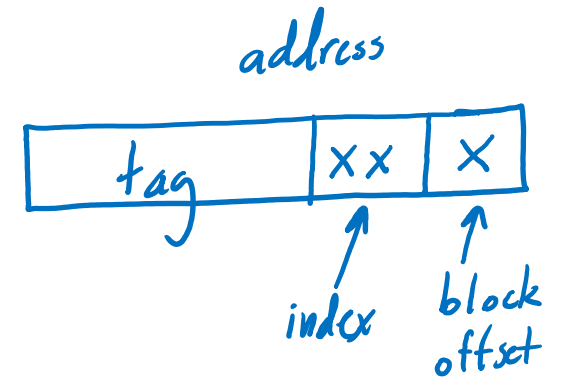
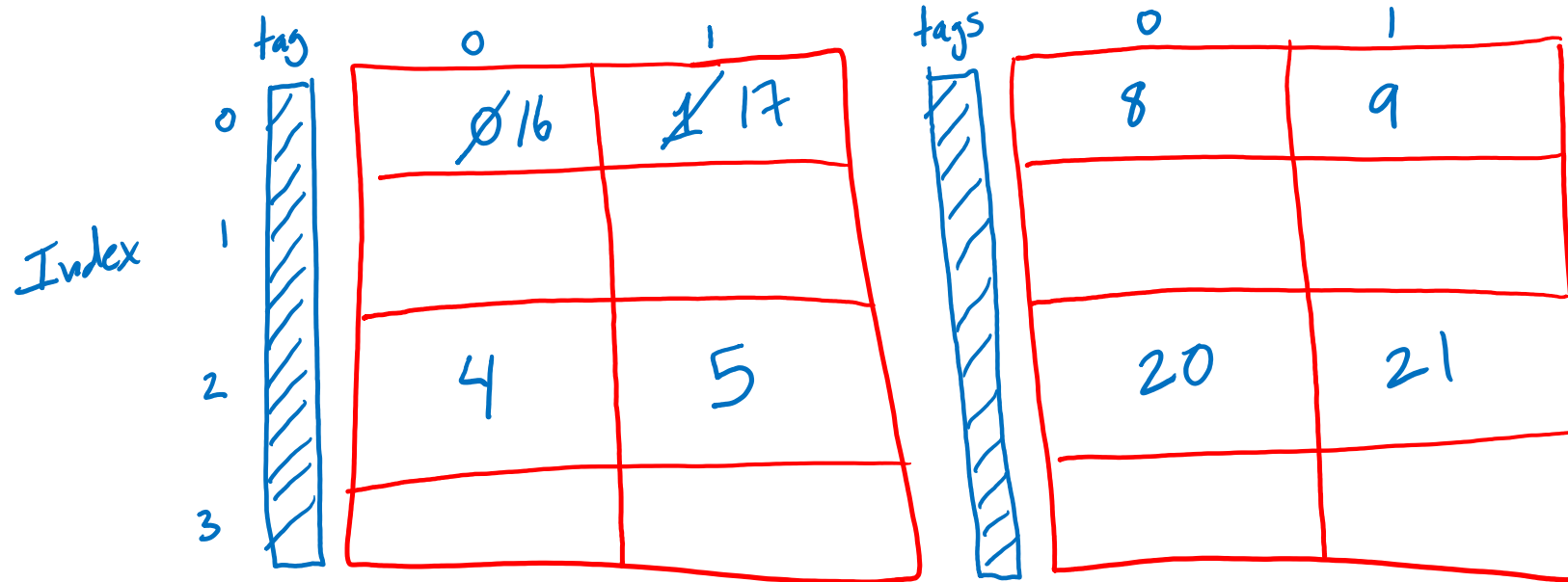
$$\text{bl. off.} = \text{addr.} \% 4$$

$$\text{index} = \underbrace{\left( \frac{\overset{17}{20}}{4} \right)}_{\text{floor}} \% 4$$



Show the hits and misses for a 2-way associative cache with a block length of 2-words and a total size of 16 words. The cache is word addressable (there is no byte-offset).

Addresses:  $\begin{matrix} M & M & M & H & M & M \\ 1 & 4 & 8 & 5 & 20 & 17 \end{matrix}$   
 $\begin{matrix} \text{bl.off.} = 1 & 0 & 0 & 1 & 0 & 1 \\ \text{index} = 0 & 2 & 0 & 2 & 2 & 0 \end{matrix}$



$$\text{bl. off.} = \text{address} \% 2$$

$$\text{index} = \left( \text{address} / 2 \right) \% 4$$

floor

- 64 KB  
 - 2-way  
 - 16 word blocks  
 byte addressable  


---

 index? tag?  
 bits?

bl. off. =  $\begin{matrix} M & M & M & H & M & \\ 1 & 4 & 8 & 5 & 20 & 17 \\ 1 & 0 & 0 & 1 & 0 & 1 \end{matrix}$

- ✓ - fully associative
- ✓ - 8 total words
- ✓ - 2 word blocks
- word addressable (no byte offset)



bl. off. = addr % 2

